

Universidade Presbiteriana Mackenzie

Faculdade de Computação e Informática - FCI

Estrutura de Dados II

Projeto Prático 1 e 2

Sistema de Autocomplete e Correção Ortográfica

Prof. Dr. Jean M. Laine

Descrição do Projeto

Período: 2026/2

Entrega Final: Conforme cronograma disponível no Moodle

São Paulo, 2026

Sumário

1	Introdução	4
2	Objetivos	4
2.1	Objetivo Geral	4
2.2	Objetivos Específicos	4
3	Estrutura do Projeto	4
4	Fase 1: Trie + Busca Simples	5
4.1	Objetivo	5
4.2	Conceitos Envolvidos	5
4.3	Requisitos Funcionais	6
4.4	Exemplo de Uso	7
4.5	Classe TrieNode	7
4.6	Classe Trie	7
5	Fase 2: BST + Ranking por Frequência	8
5.1	Objetivo	8
5.2	Conceitos Envolvidos	8
5.3	Requisitos Funcionais	9
5.4	Classe BSTWordRanking	9
5.5	Registro de Uso e Atualização de Frequência	10
5.6	Classe Trie: Métodos de Frequência	10
5.7	Exemplo de Uso	11
6	Fase 3: Correção Ortográfica e Cache	11
6.1	Objetivo	11
6.2	Conceitos Envolvidos	11
6.3	Requisitos Funcionais	12
6.3.1	Como funciona a invalidação de cache	12
6.4	Distância de Levenshtein	12
6.5	Exemplo de Uso	13
6.6	Classe Dicionario (Integração Final)	13
7	Fase 4: Integração + Relatório	13
7.1	Objetivo	13
7.2	Atividades	14
7.3	Exemplo de Integração	14
8	Exemplo de Saída Esperada	14
9	Entrada e Saída de Dados	15
9.1	Formato do Dicionário	15
9.2	Formato do Arquivo de Teste	15
9.2.1	Estrutura do Arquivo	15
9.2.2	Comandos Disponíveis	16
9.2.3	Exemplo Completo	16
9.2.4	Como Utilizar	17
9.3	Exemplo de Saída do Programa de Teste	17

10 Relatório	18
11 Critérios de Avaliação	20
11.1 1º Bimestre (Lab 1) Fases 1 e 2	20
11.2 2º Bimestre (Lab 2) Fase 3 + Fase 4	21
11.3 Relatório Parcial (1º Bimestre)	21
11.4 Relatório Final (2º Bimestre)	22
12 Organização do Trabalho em Grupo	22
12.1 Recomendações	22
12.2 Identificação no Relatório	22
13 Regulamento de Apresentação e Entrega dos Projetos Práticos	22
13.1 Apresentação e Defesa	22
14 Orientações sobre Uso de Inteligência Artificial	23
14.1 O que é Permitido	23
14.2 O que NÃO é Permitido	23
14.3 Declaração de Uso de IA no Relatório	23
14.4 Exemplo de Declaração	24
14.5 Avaliação e Integridade Acadêmica	24
15 Testes Automatizados	24
15.1 Requisitos dos Testes	24
15.2 Exemplo de Teste	25
15.3 Estrutura de Pastas Sugerida	25
16 Erros Comuns a Evitar	25
17 Material de Apoio	26

Resumo do Projeto

Contexto: Todos os dias, utilizamos sistemas que antecipam o que estamos prestes a digitar. Quando você começa a escrever no Google, no seu celular ou em um editor de texto, sugestões aparecem instantaneamente. Este comportamento é possível graças a uma estrutura de dados chamada **Trie** (também conhecida como árvore de prefixos, árvore digital ou árvore lexicográfica).

Objetivo do Projeto: Desenvolver um sistema de autocomplete e correção ortográfica, capaz de carregar um dicionário de palavras e oferecer sugestões inteligentes baseadas em prefixo, frequência de uso e similaridade.

Estruturas de Dados a Serem Utilizadas:

- **Trie:** Armazenamento e busca por prefixo.
- **Árvore Binária de Busca (BST):** Ordenação de palavras por frequência de uso.
- **HashMap:** Cache de sugestões frequentes.

Algoritmos:

- **Distância de Levenshtein:** Correção ortográfica baseada em similaridade.

Cronograma de Entregas

Fase	Conteúdo	Descrição
Fase 1	Trie + Busca Simples	Implementação da estrutura Trie e busca por palavras
Fase 2	BST + Ranking por Frequência	Árvore Binária de Busca e ordenação por frequência
Fase 3	Correção Ortográfica + Cache	Sugestões de correção e cache de resultados
Fase 4	Integração + Relatório	Integração das fases anteriores e documentação

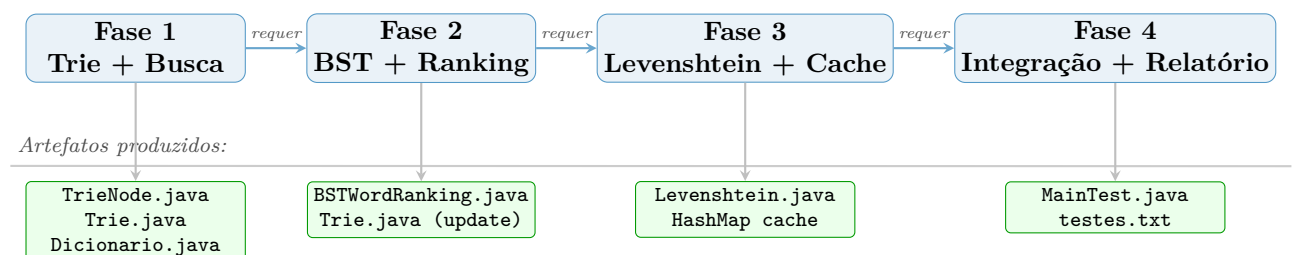


Figura 1: Dependências entre as fases do projeto. Cada fase requer a anterior e produz os artefatos listados abaixo.

Projeto 1o Bimestre

- Fase 1 ([Trie](#))
- Fase 2 ([BST](#))

Projeto 2o Bimestre

- Fase 1 ([Trie](#)) ✓
- Fase 2 ([BST](#)) ✓
- Fase 3 ([Correção + Cache](#))
- Fase 4 ([Integração + Relatório](#))

"O projeto será desenvolvido em 4 fases, permitindo que os conceitos sejam consolidados gradualmente e que o código evolua com o conteúdo abordado pela disciplina durante o semestre."

Introdução

Todos os dias, utilizamos sistemas que antecipam o que estamos prestes a digitar. Quando você começa a escrever no Google, no seu celular ou em um editor de texto, sugestões aparecem instantaneamente. Este comportamento é possível graças a uma estrutura de dados chamada **Trie** (ou árvore de prefixos).

Além disso, quando cometemos um erro de digitação, o sistema nos oferece correções. O teclado do seu celular (como o Gboard da Google, o teclado padrão da Apple ou da Samsung) fazem isso. Por trás disso, está o cálculo da **Distância de Levenshtein**, que mede a similaridade entre palavras.

Neste projeto, você irá construir um sistema completo de autocomplete e correção ortográfica, aplicando conceitos fundamentais de Estruturas de Dados:

- **Trie** para armazenamento e busca por prefixos.
- **Árvore Binária de Busca** (BST) para ordenar palavras por frequência de uso.
- **HashMap** para cache de sugestões frequentes.
- **Distância de Levenshtein** para correção ortográfica.

O projeto será desenvolvido em **4 fases**, permitindo que os conceitos sejam consolidados gradualmente e que o código evolua com o conteúdo do semestre.

Objetivos

Objetivo Geral

Desenvolver um sistema de autocomplete e correção ortográfica, capaz de carregar um dicionário de palavras e oferecer sugestões inteligentes baseadas em prefixo, frequência de uso e similaridade.

Objetivos Específicos

- Estudar, compreender e implementar uma **Trie** para armazenar e buscar palavras por prefixo.
- Utilizar uma **Árvore Binária de Busca** para ordenar palavras por frequência de uso.
- Empregar um **HashMap** como cache para otimizar consultas frequentes.
- Implementar o algoritmo de **Distância de Levenshtein** para correção ortográfica.
- Trabalhar com arquivos externos para carregar dicionários.
- Produzir um relatório com análises de complexidade e testes de desempenho.

Estrutura do Projeto

O projeto é dividido em **4 fases**, cada uma com entregas parciais. Cada fase introduz novas estruturas de dados e funcionalidades, permitindo que o código seja construído de forma incremental.

Fase	Título	Conceitos	Entrega
1	Trie + Busca Simples	Trie, Percursos, Arquivos	1o Bimestre/N1
2	BST + Ranking por Frequência	BST, Frequência, Ordenação	1o Bimestre/N1
3	Correção + Cache	Levenshtein, HashMap	2o Bimestre/N2
4	Integração + Relatório	Integração, Testes, Documentação	2o Bimestre/N2

Tabela 1: Fases do projeto

Fase 1: Trie + Busca Simples

Objetivo

Implementar a estrutura base do sistema: a **Trie**. O sistema deve ser capaz de carregar em memória um dicionário de palavras armazenado em um arquivo, buscar palavras e listar todas as palavras que começam com um determinado prefixo.

Conceitos Envolvidos

- **Trie:** Árvore de prefixos para armazenamento eficiente de palavras.
- **Percursos:** DFS para listar palavras. DFS é a sigla para Depth-First Search, ou Busca em Profundidade. É um algoritmo usado para explorar ou buscar elementos em estruturas de dados de grafos e árvores. A regra central do DFS é: vá o mais fundo possível em uma ramificação antes de retroceder (fazer o backtracking).
- **Arquivos:** Leitura de dicionário a partir de um arquivo de texto.

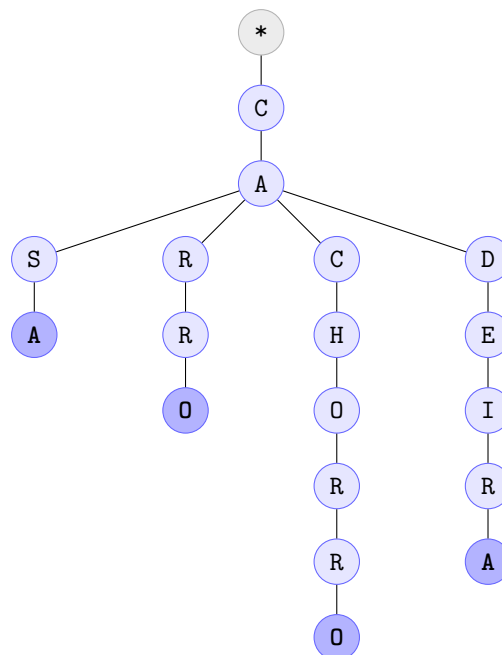


Figura 2: Exemplo de estrutura de uma Trie contendo as palavras: **CASA**, **CARRO**, **CA-CHORRO** e **CADEIRA**

A Figura 2 ilustra a estrutura de uma Trie de exemplo. Observe que:

- A raiz é vazia (representada por *).
- Cada nó representa uma letra.
- Nós em azul escuro (●) indicam o **final de uma palavra**. Nestes nós, além da flag `isEndOfWord = true`, também é armazenada a **frequência** da palavra (atributo `int frequency`), que será utilizada na Fase 2 para ordenar as sugestões.
- Palavras compartilham prefixos comuns (ex: "CA" é compartilhado por todas as palavras).

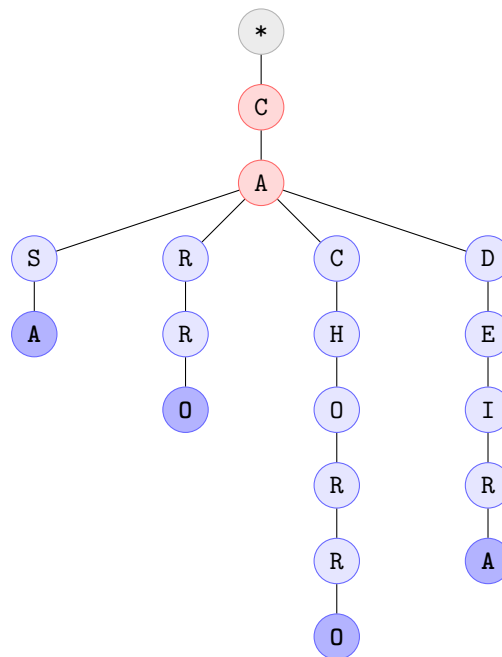


Figura 3: Busca pelo prefixo "CA" (nós percorridos em vermelho)

A Figura 3 demonstra a busca pelo prefixo "CA". O percurso (nós em vermelho) é feito letra por letra:

1. Começa na raiz.
2. Vai para o nó 'C'.
3. Vai para o nó 'A'.
4. A partir deste ponto, todas as palavras que começam com "CA" estão na subárvore que tem o nó "A" como raiz.

Requisitos Funcionais

1. **Carregar Dicionário:** O programa deve ler um arquivo de texto contendo uma palavra por linha e carregá-lo na Trie.
2. **Inserir Palavra:** Implementar o método para inserir uma palavra na Trie.
3. **Buscar Palavra:** Implementar o método para verificar se uma palavra existe no dicionário.
4. **Buscar por Prefixo:** Implementar o método para verificar se existe alguma palavra com um determinado prefixo.

5. **Sugerir Palavras:** Implementar o método que recebe um prefixo e retorna uma lista de todas as palavras do dicionário que começam com esse prefixo (em ordem alfabética).

Exemplo de Uso

Listing 1: Exemplo de uso da Fase 1

```
Dicionario dic = new Dicionario();
dic.carregar("palavras.txt");

System.out.println(dic.buscar("casa"));           // true
System.out.println(dic.buscar("cade"));          // false
System.out.println(dic.buscarPrefixo("cade"));    // true

List<String> sugestoes = dic.sugerir("comp");
// [companhia, compartilhar, compra, computador]
```

Classe TrieNode

Listing 2: Estrutura do nó da Trie

```
class TrieNode {
    private static final int ALPHABET_SIZE = 26; // a-z

    TrieNode[] children; // Array de filhos (um por letra)
    boolean isEndOfWord; // Marca o final de uma palavra
    int frequency;        // Frequencia de uso (sera usada na Fase 2)

    public TrieNode() {
        children = new TrieNode[ALPHABET_SIZE];
        isEndOfWord = false;
        frequency = 0;
    }
}
```

Classe Trie

Listing 3: Métodos principais da Trie

```
public class Trie {
    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    // Insere uma palavra na Trie
    public void insert(String word) { ... }

    // Verifica se uma palavra existe
    public boolean search(String word) { ... }

    // Verifica se existe alguma palavra com o prefixo
    public boolean startsWith(String prefix) { ... }

    // Retorna todas as palavras com o prefixo
```

```

public List<String> suggest(String prefix) { ... }

// Metodo auxiliar para coletar palavras
private void collectWords(TrieNode node, StringBuilder prefix,
List<String> results) { ... }
}

```

Fase 2: BST + Ranking por Frequência

Objetivo

Evoluir o sistema para que as sugestões não sejam apenas alfabéticas, mas **ordenadas por frequência de uso**. Para isso, você irá implementar uma **Árvore Binária de Busca (BST)** que armazena palavras e suas frequências, permitindo consultas eficientes.

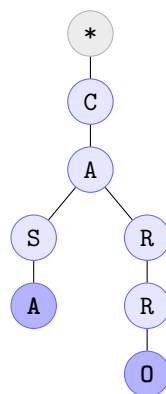


Figura 4: Trie (compartilha prefixos)

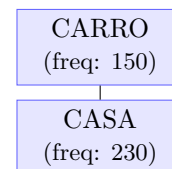


Figura 5: BST (armazena palavras e frequências)

Figura 6: Comparação entre Trie e BST para armazenar as mesmas palavras (exemplo simplificado)

A Figura 6 compara as duas estruturas:

- **Trie:** Compartilha prefixos, eficiente para buscas por prefixo.
- **BST:** Armazena palavras completas, eficiente para buscas exatas e ordenação.

Neste projeto, as duas estruturas trabalham juntas:

1. A **Trie** encontra rapidamente todas as palavras com um determinado prefixo.
2. A **BST** ordena essas palavras por frequência de uso.

Conceitos Envolvidos

- **Árvore Binária de Busca:** Estrutura para armazenar e ordenar palavras.
- **Frequência de Uso:** Contagem de quantas vezes uma palavra foi usada.
- **Atualização de Dados:** Incrementar frequência quando uma palavra é usada.
- **Ordenação:** Percorrer a BST para listar palavras por frequência.

Requisitos Funcionais

1. **Registrar Uso:** Método que recebe uma palavra e incrementa sua frequência de uso na BST e na Trie.
2. **Atualizar Trie:** A Trie deve armazenar a frequência de cada palavra em seu nó final, permitindo consultas rápidas de frequência por palavra.
3. **Manter BST Atualizada:** A BST também deve ser atualizada sempre que uma palavra for usada, mantendo o ranking de palavras mais frequentes.
4. **Sugerir Ordenado por Frequência:** Método que retorna palavras ordenadas da mais frequente para a menos frequente.

Classe BSTWordRanking

A BST armazena cada palavra única junto com sua frequência. O critério de ordenação é **lexicográfico** (para permitir busca eficiente), mas as palavras podem ser recuperadas em ordem de frequência.

Listing 4: Estrutura da BST para ranking

```
public class BSTWordRanking {
    private Node root;

    private class Node {
        String word;
        int frequency;
        Node left, right;

        Node(String word, int frequency) {
            this.word = word;
            this.frequency = frequency;
        }
    }

    // Insere palavra nova OU atualiza frequencia se ja existir
    public void addOrUpdate(String word, int newFrequency) { ... }

    // Retorna a frequencia de uma palavra (0 se nao existir)
    public int getFrequency(String word) { ... }

    // Remove uma palavra (usado apenas internamente)
    public void remove(String word) { ... }

    // Retorna as N palavras mais frequentes (ordem decrescente)
    public List<String> getTopWords(int n) { ... }

    // Coleta todas as palavras em ordem (auxiliar)
    private void collectAll(Node node, List<Node> out) { ... }
}
```

Decisão de projeto: A BST é ordenada por **ordem lexicográfica** (não por frequência), porque:

- Permite busca eficiente $O(h)$ por palavra.
- Facilita atualização de frequência (busca + atualização do valor).
- Para extrair "top N", percorremos toda a árvore e ordenamos a lista final por frequência ($O(n \log n)$).

Registro de Uso e Atualização de Frequência

Quando o usuário usa uma palavra (por exemplo, aceitando uma sugestão), sua frequência deve ser incrementada. O método abaixo mantém Trie e BST sincronizadas:

Listing 5: Registrando uso de uma palavra

```
// Em Dicionario.java
public void registrarUso(String palavra) {
    // 1. Descobre a frequencia atual (a partir da Trie)
    int freqAtual = trie.getFrequency(palavra);

    // 2. Se a palavra nao existe, insere com frequencia 0
    if (freqAtual == 0 && !trie.search(palavra)) {
        trie.insert(palavra);
    }

    // 3. Incrementa e propaga para as duas estruturas
    int novaFreq = freqAtual + 1;
    trie.updateFrequency(palavra, novaFreq);
    ranking.addOrUpdate(palavra, novaFreq);

    // 4. Invalida cache de todos os prefixos da palavra
    invalidateCacheFor(palavra);
}
```

Observações importantes:

- `addOrUpdate(palavra, novaFreq)` **atualiza** o valor na BST se a palavra já existir, ou **insere** um novo nó caso contrário.
- A Trie é a **única fonte de verdade** para a frequência; a BST é mantida em sincronia por conveniência de ordenação.
- A invalidação do cache está detalhada na Seção 6.2.

Classe Trie: Métodos de Frequência

Para suportar a Fase 2, a Trie precisa de dois métodos adicionais:

Listing 6: Metodos de frequencia na Trie

```
public class Trie {
    private TrieNode root;

    // ... insert, search, startsWith, suggest (Fase 1) ...

    // Retorna a frequencia da palavra (0 se nao existir ou nao for final)
    public int getFrequency(String word) {
        TrieNode node = traverseTo(word);
        return (node != null && node.isEndOfWord) ? node.frequency : 0;
    }

    // Atualiza a frequencia de uma palavra existente
    public void updateFrequency(String word, int frequency) {
        TrieNode node = traverseTo(word);
        if (node != null && node.isEndOfWord) {
            node.frequency = frequency;
        }
    }
}
```

```

    }

    // Auxiliar: percorre a Trie ate o no correspondente a palavra
    private TrieNode traverseTo(String word) {
        TrieNode current = root;
        for (int i = 0; i < word.length(); i++) {
            int idx = word.charAt(i) - 'a';
            if (idx < 0 || idx >= 26 || current.children[idx] == null) {
                return null;
            }
            current = current.children[idx];
        }
        return current;
    }
}

```

Importante: `traverseTo()` **não** verifica se o nó é final de palavra isso é responsabilidade dos chamadores (`getFrequency` e `updateFrequency` fazem essa checagem explicitamente).

Exemplo de Uso

Listing 7: Exemplo da Fase 2

```

Dicionario dic = new Dicionario();
dic.carregar("palavras.txt");

// Usuario usa palavras
dic.registrarUso("computador"); // freq = 1
dic.registrarUso("computador"); // freq = 2
dic.registrarUso("casa");       // freq = 1

// Sugestoes ordenadas por frequencia
List<String> sugestoes = dic.sugerir("comp");
// [computador (freq: 2), compra (freq: 0), ...]

```

Fase 3: Correção Ortográfica e Cache

Objetivo

Adicionar duas funcionalidades avançadas: **correção ortográfica** (usando Distância de Levenshtein) e **cache de sugestões** (usando `HashMap` para armazenar resultados de prefixos comuns).

Conceitos Envolvidos

- **Distância de Levenshtein:** Medida de similaridade entre strings.
- **HashMap:** Estrutura de chave-valor para cache de resultados.
- **Otimização:** Reduzir tempo de resposta para consultas repetidas.
- **Programação Dinâmica:** Implementação do algoritmo de Levenshtein.

Requisitos Funcionais

1. **Correção Ortográfica:** Método que recebe uma palavra e retorna a palavra mais semelhante do dicionário usando a Distância de Levenshtein.
2. **Limitar Busca (Opcional):** A correção pode ser otimizada usando a Trie para podar candidatos com diferença de comprimento maior que o `maxDistance` desejado. Esta é uma **atividade opcional** para grupos que desejam explorar otimizações.
3. **Cache de Sugestões:** Armazenar em um `HashMap` os resultados de sugestões para prefixos comuns.
4. **Invalidar Cache:** Quando uma palavra é adicionada ou sua frequência muda, o cache deve ser invalidado para **todos os prefixos dessa palavra**.

6.3.1 Como funciona a invalidação de cache

Para a palavra "computador" de comprimento $k = 10$, os prefixos afetados são exatamente:

c, co, com, comp, compu, comput, computa, computad, computado, computador

Ou seja, há k prefixos afetados. O algoritmo é:

Listing 8: Invalidacao de cache por prefixo

```
// Em Dicionario.java
private void invalidateCacheFor(String palavra) {
    // Remove do cache todos os prefixos da palavra
    for (int i = 1; i <= palavra.length(); i++) {
        String prefixo = palavra.substring(0, i);
        cache.remove(prefixo);
    }
}
```

Complexidade: $O(k^2)$ por palavra, onde k é o comprimento (a construção de cada substring custa $O(i)$ e são k substrings). Para palavras curtas ($k \leq 30$), isso é irrelevante.

Alternativa mais eficiente: invalidar todo o cache quando uma palavra é usada ($O(1)$, mas perde todos os ganhos). Fica a critério do aluno escolher.

Distância de Levenshtein

Complexidade: $O(m \cdot n)$, onde m e n são os comprimentos das duas palavras.

Listing 9: Implementacao da distancia de Levenshtein

```
public static int levenshtein(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];

    for (int i = 0; i <= a.length(); i++) dp[i][0] = i;
    for (int j = 0; j <= b.length(); j++) dp[0][j] = j;

    for (int i = 1; i <= a.length(); i++) {
        for (int j = 1; j <= b.length(); j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],
                    Math.min(dp[i - 1][j], dp[i][j - 1]));
            }
        }
    }
}
```

```

    }
}
return dp[a.length()][b.length()];
}

```

Exemplo de Uso

Listing 10: Exemplo de correcao ortografica

```

Dicionario dic = new Dicionario();
dic.carregar("palavras.txt");

// Correcao ortografica
String corrigida = dic.corrigir("computadro");
System.out.println(corrigida); // "computador"

// Cache de sugestoes
List<String> sugestoes = dic.sugerirComCache("comp");
// Resultado e armazenado no cache
// Proxima consulta com "comp" e instantanea (O(1))

```

Classe Dicionario (Integração Final)

Listing 11: Classe principal integrando todas as estruturas

```

public class Dicionario {
    private Trie trie;
    private BSTWordRanking ranking;
    private Map<String, List<String>> cache;

    public Dicionario() {
        trie = new Trie();
        ranking = new BSTWordRanking();
        cache = new HashMap<>();
    }

    public void carregar(String arquivo) { ... }
    public List<String> sugerir(String prefixo) { ... }
    public List<String> sugerirComCache(String prefixo) { ... }
    public void registrarUso(String palavra) { ... }
    public String corrigir(String palavra) { ... }
}

```

Observação: Ao carregar o dicionário, cada palavra é inserida na Trie com **frequency** = 0 e registrada na BST com **frequency** = 0. A frequência só é incrementada quando o usuário interage com a palavra (via **registrarUso**).

Fase 4: Integração + Relatório

Objetivo

Consolidar todo o trabalho desenvolvido nas fases anteriores, integrando as estruturas em um sistema único e produzindo um relatório completo com análises de desempenho e complexidade.

Atividades

1. Integração das Estruturas:

- Unir todas as classes em um único sistema funcional.
- Garantir que as operações de busca, sugestão, correção e cache funcionem em conjunto.
- Validar o sistema com o arquivo de testes `testes.txt`.

2. Testes Automatizados:

- Implementar a classe `MainTest.java` com todos os testes.
- Cada teste deve exibir `[PASSED]` ou `[FAILED]`.
- O programa deve retornar o resultado para cada teste.

3. Relatório Final:

- Consolidar as análises de complexidade de todas as estruturas.
- Preencher e analisar as tabelas de desempenho (Seção 10).
- Documentar desafios, aprendizados e possíveis melhorias.
- Incluir a Declaração de Uso de IA (Seção 14).

Exemplo de Integração

Listing 12: Exemplo do sistema integrado

```
Dicionario dic = new Dicionario();
dic.carregar("palavras.txt");

// Busca e registro de frequencia
dic.registrarUso("computador");
dic.registrarUso("computador");

// Sugestao com cache
List<String> sugestoes = dic.sugerirComCache("comp");
System.out.println(sugestoes);
// [computador(2), compra(0), companhia(0), compartilhar(0)]

// Correcao ortografica
String corrigida = dic.corrigir("computadro");
System.out.println(corrigida); // "computador"
```

Exemplo de Saída Esperada

```
=== Dicionario Carregado ===
Palavras carregadas: 50000

=== Sugestao Simples ===
Prefixo: "comp"
Sugestoes: [companhia, compartilhar, compra, computador]

=== Sugestao com Ranking (apos varias buscas) ===
Prefixo: "comp"
```

```
computador    (freq: 42)
compra        (freq: 18)
compartilhar  (freq: 5)
companhia      (freq: 2)

=== Correcao Ortografica ===
Palavra digitada: "computadro"
Correcao: computador
Distancia: 2

=== Cache de Sugestoes ===
Primeira consulta: 5ms
Segunda consulta (cache): 0ms
```

Entrada e Saída de Dados

Formato do Dicionário

O dicionário deve ser um arquivo de texto com uma palavra por linha:

```
casa
carro
cachorro
cadeira
computador
comunidade
...
```

Formato do Arquivo de Teste

Para facilitar a validação do sistema, você deve criar um arquivo `testes.txt` com os comandos a serem executados. O arquivo deve seguir o formato descrito abaixo.

9.2.1 Estrutura do Arquivo

Cada linha do arquivo contém um comando no formato:

```
<comando> <argumentos> // ou <comando> <argumentos> -> <saida_esperada>
```

Onde:

- <comando> é uma das palavras-chave listadas abaixo.
- <argumentos> são os parâmetros do comando.
- -> <saida_esperada> (opcional) indica o resultado esperado para validação automática.

9.2.2 Comandos Disponíveis

Comando	Formato	Descrição
add	add <palavra>	Insere palavra no dicionário (freq = 0).
use	use <palavra>	Registra uso da palavra (+1 na frequência).
search	search <palavra>	Verifica se a palavra existe (true/false).
suggest	suggest <prefixo>	Retorna a lista de palavras com o prefixo.
correct	correct <palavra>	Retorna a correção ortográfica sugerida.

Tabela 2: Comandos disponíveis para o arquivo de teste

9.2.3 Exemplo Completo

```
# =====
# ARQUIVO DE TESTE - AUTOCOMPLETE
# =====
# Comandos disponiveis:
#   add <palavra>      - insere palavra no dicionario
#   use <palavra>      - registra uso (+1 na frequencia)
#   search <palavra>   - busca palavra (true/false)
#   suggest <prefixo>  - lista palavras com o prefixo
#   correct <palavra>  - corrige ortograficamente
# =====

# --- FASE 1: Insercao e Busca ---
add casa
add carro
add cachorro
add cadeira
add computador
add compra

search casa      -> true
search carro     -> true
search cadeira   -> true
search computador -> true
search xyz       -> false

# --- FASE 1: Sugestao por Prefixo ---
suggest ca       -> [cachorro, cadeira, carro, casa]
suggest comp     -> [companhia, compartilhar, compra, computador]
suggest xyz      -> []

# --- FASE 2: Ranking por Frequencia ---
use computador   # +1 (freq = 1)
use computador   # +1 (freq = 2)
```

```

use casa                # +1 (freq = 1)

suggest comp            -> [computador(2), compra(0), companhia(0), compartilhar(0)]

# --- FASE 3: Correcao Ortografica ---
correct computadro     -> computador
correct caza           -> casa
correct cadiera        -> cadeira

```

9.2.4 Como Utilizar

1. Crie um arquivo `testes.txt` com os comandos desejados.
2. Execute o programa com o arquivo de teste:

```
java -cp . Dicionario testes.txt
```

3. O programa deve processar cada linha, executar o comando e exibir o resultado.

Exemplo de Saída do Programa de Teste

Ao executar o programa com o arquivo `testes.txt`, a saída esperada é:

```

=== Processando arquivo de teste: testes.txt ===

[1] add casa                -> OK
[2] add carro               -> OK
[3] add cachorro            -> OK
[4] add cadeira             -> OK
[5] add computador          -> OK
[6] add compra              -> OK

[7]  search casa            -> true    [PASSED]
[8]  search carro           -> true    [PASSED]
[9]  search cadeira         -> true    [PASSED]
[10] search computador       -> true    [PASSED]
[11] search xyz             -> false   [PASSED]

[12] suggest ca              -> [cachorro, cadeira, carro, casa] [PASSED]
[13] suggest comp            -> [companhia, compartilhar, compra, computador] [PASSED]
[14] suggest xyz             -> []    [PASSED]

[15] use computador          -> OK (freq: 1)
[16] use computador          -> OK (freq: 2)
[17] use casa                -> OK (freq: 1)

[18] suggest comp            -> [computador(2), compra(0), companhia(0), compartilhar(0)] [PA

[19] correct computadro     -> computador [PASSED]
[20] correct caza           -> casa        [PASSED]
[21] correct cadiera        -> cadeira     [PASSED]

```

=== RESULTADO: 14/14 testes PASSED ===

Relatório

O relatório deve conter as seguintes seções:

1. **Introdução:** Contextualização do projeto e dos conceitos utilizados.
2. **Descrição das Estruturas:** Explicação detalhada da Trie, BST e HashMap.
3. **Análise de Complexidade**

No relatório, você deve incluir uma análise de complexidade para as principais operações do sistema. Para cada estrutura e operação listada abaixo, responda:

- Qual é a complexidade de tempo (Big O) da operação?
- Explique o porquê dessa complexidade, considerando a implementação utilizada.
- Há diferença entre o pior caso, caso médio e melhor caso?

Estruturas e operações a serem analisadas:

(a) **Trie:**

- Inserção de uma palavra.
- Busca de uma palavra.
- Busca por prefixo.
- Sugestão de palavras a partir de um prefixo (considerando o custo de percorrer a subárvore).

(b) **Árvore Binária de Busca (BST):**

- Inserção de um nó (palavra + frequência).
- Busca de uma palavra (por frequência).
- Atualização de frequência.
- Obtenção das n palavras mais frequentes.

(c) **HashMap (Cache):**

- Inserção de uma sugestão no cache.
- Busca de uma sugestão no cache.

(d) **Distância de Levenshtein:**

- Cálculo da distância entre duas palavras.
- Como a Trie pode ser usada para otimizar a correção ortográfica?

Estrutura	Operação	Complexidade Esperada
Trie	Inserção	$O(k)$
Trie	Busca	$O(k)$
Trie	Sugestão por prefixo	$O(k + m)$
BST	Inserção	$O(h)$, h = altura
BST	Busca	$O(h)$
BST	Atualização de freq.	$O(h)$
BST	Top-N (ingênuo)	$O(n \log n)$
HashMap	Insert/Lookup	$O(1)$ médio, $O(n)$ pior
Levenshtein	Cálculo	$O(mn)$

Tabela 3: Complexidades esperadas (referência para o relatório)

Tabela de Referência (Gabarito Esperado) Formato esperado de resposta:

Estrutura: Trie

Operacao: Insercao

Complexidade: $O(k)$, onde k e o comprimento da palavra.

Justificativa: Percorremos a Trie letra por letra, criando novos nos quando necessario. Cada operacao (criar no ou acessar filho) e $O(1)$, portanto o custo total e proporcional ao numero de caracteres.

4. Testes de Desempenho

Para cada um dos testes listados abaixo, preencha as tabelas com os resultados obtidos e inclua uma análise textual discutindo os achados.

Teste 1: Tempo de Sugestão para Diferentes Prefixos Meça o tempo (em milissegundos) para obter sugestões de diferentes prefixos, utilizando dicionários de diferentes tamanhos.

Prefixo	1.000 palavras	10.000 palavras	50.000 palavras	100.000 palavras
"a"				
"ab"				
"c"				
"ca"				
"z"				

Tabela 4: Tempo de sugestão (ms) para diferentes prefixos e tamanhos de dicionário

Análise esperada:

- O tempo de sugestão é proporcional ao número de palavras encontradas?
- Prefixos mais raros (ex: "z") são mais rápidos que prefixos comuns (ex: "a")?
- Como o tamanho do dicionário afeta o tempo de sugestão?

Teste 2: Comparação entre BST e HashMap para Cache Compare o tempo de resposta para sugestões com e sem cache, utilizando um cenário com consultas repetidas.

Prefixo	Sem Cache (ms)	Com Cache (ms)	Ganho (%)
"a"			
"c"			
"ca"			
"comp"			
"z"			

Tabela 5: Comparação de tempo de resposta com e sem cache

Análise esperada:

- Qual é o ganho percentual médio com o uso do cache?
- Há diferença no ganho para prefixos mais longos?
- Em que situações o cache é mais vantajoso?

Teste 3: Tempo de Correção Ortográfica Meça o tempo necessário para corrigir palavras com diferentes níveis de distância de Levenshtein.

Palavra	Correção Esperada	Distância	Tempo (ms)
"computadro"	"computador"	2	
"caza"	"casa"	1	
"cadiera"	"cadeira"	2	
"caro"	"carro"	1	
"xyz"	(nenhuma)	-	

Tabela 6: Tempo de correção ortográfica para diferentes palavras

Análise esperada:

- Como a distância de Levenshtein afeta o tempo de correção?
- A Trie ajudou a limitar o número de candidatos?
- Qual foi o pior caso encontrado?

5. **Conclusão:** Aprendizados, dificuldades e possíveis melhorias.

Critérios de Avaliação

O projeto será avaliado em dois bimestres, conforme a distribuição abaixo.

1º Bimestre (Lab 1) Fases 1 e 2

Valor total: 10,0 pontos

Critério	Peso	Descrição
Fase 1 - Trie	2,5	Implementação correta da Trie, inserção, busca, sugestões.
Fase 2 - BST	2,5	Implementação da BST, ranking por frequência, atualização.
Qualidade do Código (Fases 1 e 2)	1,5	Organização, comentários, nomenclatura.
Relatório Parcial (Fases 1 e 2)	3,5	Análises, testes, discussão.

Tabela 7: Avaliação do 1º Bimestre (Lab 1)

2º Bimestre (Lab 2) Fase 3 + Fase 4**Valor total: 10,0 pontos**

Critério	Peso	Descrição
Fase 3 - Correção + Cache	2,5	Levenshtein correto, cache com HashMap.
Fase 4 - Integração	2,5	Sistema completo funcionando, testes automatizados.
Qualidade do Código (Fases 3 e 4)	1,5	Organização, comentários, nomenclatura.
Relatório Final	3,5	Análise completa, testes de desempenho, discussão.

Tabela 8: Avaliação do 2º Bimestre (Lab 2)

Relatório Parcial (1º Bimestre)

O relatório deve conter as seguintes seções:

- 1. Introdução:** Contextualização do projeto.
- 2. Descrição das Estruturas (Fases 1 e 2):**
 - Implementação da Trie (inserção, busca, sugestão).
 - Implementação da BST (rankings, frequência).
 - Análise de complexidade de cada operação.
- 3. Testes Iniciais:** Validação das funcionalidades implementadas.
- 4. Conclusão Parcial:** Dificuldades e aprendizados.

Entrega: PDF com até 6 páginas.

Relatório Final (2º Bimestre)

O relatório final deve consolidar todo o trabalho, incluindo a Fase 3, Fase 4 e a integração do sistema.

1. **Introdução:** Contextualização e objetivos do projeto.
2. **Descrição Completa das Estruturas:**
 - Trie (Fase 1)
 - BST (Fase 2)
 - HashMap (Cache) e Levenshtein (Fase 3)
3. **Análise de Complexidade:** Tabela com Big O de todas as operações.
4. **Testes de Desempenho:**
 - Tabelas e gráficos com tempos de resposta.
 - Comparação BST vs HashMap para cache.
 - Análise da correção ortográfica.
5. **Declaração de Uso de IA:** Conforme diretrizes da Seção 14.
6. **Conclusão:** Desafios, aprendizados e possíveis melhorias.

Entrega: PDF com até 6 páginas.

Organização do Trabalho em Grupo

O projeto deve ser realizado em grupos de **3 ou 4 alunos**. A formação dos grupos deve ser realizada o mais breve possível e informado ao professor.

Recomendações

- **Divisão:** Distribua as fases entre os membros.
- **Comunicação:** Mantenha contato frequente.
- **Versionamento:** Sugestão, use Git/GitHub para controle de versão.
- **Revisão:** Revise o código dos colegas.

Identificação no Relatório

Inclua no relatório os nomes e Matrícula de todos os integrantes.

Regulamento de Apresentação e Entrega dos Projetos Práticos

Apresentação e Defesa

- A apresentação do projeto é obrigatória para a atribuição de nota. Todos os integrantes devem estar presentes para a defesa do trabalho.

- Cada aluno deve demonstrar domínio sobre a implementação, sendo capaz de explicar a lógica, justificar as decisões tomadas e realizar alterações ou correções solicitadas pelo professor no momento da apresentação, se necessário.
- A ausência na apresentação ou o desconhecimento do funcionamento do código por qualquer integrante resultará em nota zero no projeto para o mesmo.

Orientações sobre Uso de Inteligência Artificial

O uso de ferramentas de Inteligência Artificial (IA) generativa, como ChatGPT, GitHub Copilot, entre outras, é permitido neste projeto, desde que observadas as diretrizes abaixo. O objetivo é que você utilize a IA como uma ferramenta de apoio ao aprendizado, e não como substituta do seu próprio raciocínio e desenvolvimento.

O que é Permitido

- **Assistência na escrita:** Utilizar IA para revisão gramatical, sugestões de estilo e formatação de texto.
- **Esclarecimento de dúvidas:** Consultar IA para entender conceitos, sintaxe de Java ou depuração de erros.
- **Geração de esboços:** Pedir à IA que sugira estruturas ou ideias iniciais para partes do código ou do relatório.
- **Tradução:** Utilizar IA para auxiliar na tradução de textos ou na compreensão de materiais em outros idiomas.

O que NÃO é Permitido

- **Geração automática de código:** Copiar e colar código gerado por IA sem compreendê-lo e sem adaptá-lo ao seu projeto. O código final deve ser de sua autoria.
- **Elaboração do relatório:** Gerar o relatório inteiro com IA e entregá-lo sem revisão e sem contribuição própria.
- **Plágio:** Apresentar como seu o trabalho gerado por IA, sem atribuição ou sem o devido processo de aprendizado.
- **Quebra de sigilo:** Inserir dados sensíveis, informações pessoais ou códigos completos do projeto em ferramentas de IA que armazenam dados.

Declaração de Uso de IA no Relatório

O relatório deve conter uma seção (ou subseção) intitulada "**Declaração de Uso de Ferramentas de IA**", onde o aluno ou grupo deve descrever:

1. **Ferramentas utilizadas:** Quais ferramentas de IA foram usadas (ex: ChatGPT, Copilot, etc.).
2. **Finalidade:** Para que foram usadas (ex: revisão de código, sugestão de estrutura, correção gramatical).
3. **Exemplos de Prompts:** Incluir os principais comandos (*prompts*) utilizados.
4. **Contribuição própria:** Declarar que o trabalho final é de sua autoria e que a IA foi usada apenas como ferramenta de apoio.

Exemplo de Declaração

Declaração de Uso de Ferramentas de IA

Durante o desenvolvimento deste projeto, utilizamos as seguintes ferramentas de IA:

1. ChatGPT: Utilizado para revisão gramatical e sugestões de estilo no relatório, e para esclarecer dúvidas sobre a sintaxe de Java.

Principais prompts: "Revise este parágrafo para melhorar a clareza", "Como implementar um método de ordenação em Java?".

2. GitHub Copilot: Utilizado como assistente de código para sugestões de autocompletar e agilizar a escrita de trechos repetitivos.

Declaramos que todo o código e a análise apresentados são de nossa autoria, tendo a IA sido utilizada apenas como ferramenta auxiliar.

Avaliação e Integridade Acadêmica

A avaliação do projeto considerará não apenas o resultado final, mas também o processo de desenvolvimento. O não cumprimento das diretrizes acima poderá ser considerado violação da integridade acadêmica, sujeitando o grupo às penalidades previstas no regimento da universidade.

A transparência no uso de IA é valorizada. A declaração de uso não reduzirá a nota; pelo contrário, demonstra maturidade e responsabilidade acadêmica.

Testes Automatizados

Para facilitar a validação do projeto e a correção automatizada, todos os grupos devem implementar e entregar uma suíte de testes simples, sem dependências externas.

Requisitos dos Testes

1. **Classe de Teste:** O projeto deve conter uma classe `MainTest.java` que implementa todos os testes.
2. **Sem Dependências:** A classe `MainTest` não pode depender de bibliotecas externas (JUnit, etc.).
3. **Cobertura:** A classe deve testar:
 - Inserção e busca na Trie
 - Busca por prefixo (sugestões)
 - Ranking por frequência (BST)
 - Integração entre Trie e BST (Dicionário)
 - Correção ortográfica (Levenshtein)
 - Cache de sugestões (HashMap)
4. **Script de Teste:** O projeto deve incluir um script executável (`run_tests.sh` para Linux/Mac ou `run_tests.bat` para Windows).

Exemplo de Teste

Listing 13: Exemplo de teste na classe MainTest

```
private static void testTrieSuggest() {
    System.out.println("[TESTE]_Busca_por_Prefixo");

    Trie trie = new Trie();
    trie.insert("casa");
    trie.insert("carro");
    trie.insert("cachorro");

    List<String> result = trie.suggest("ca");
    boolean passed = result.contains("casa") &&
        result.contains("carro");

    System.out.println(passed ? "[PASSED]" : "[FAILED]");
}
```

Estrutura de Pastas Sugerida

```
projeto/
|-- src/
|   |-- Trie.java
|   |-- TrieNode.java
|   |-- BSTWordRanking.java
|   |-- Dicionario.java
|   |-- Levenshtein.java
|   +-- MainTest.java
|-- data/
|   |-- palavras.txt
|   +-- testes.txt
|-- run_tests.sh
|-- run_tests.bat
+-- README.md
```

Erros Comuns a Evitar

- **Trie:** esquecer de marcar `isEndOfWord = true` no final.
- **Trie:** inserir palavra duplicada sem incrementar frequência.
- **BST:** comparar `String` com `==` em vez de `.equals()` ou `compareTo()`.
- **BST:** não remover o nó antigo antes de reinserir com nova frequência.
- **Cache:** esquecer de invalidar quando uma palavra é usada.
- **Levenshtein:** inicializar a matriz com tamanho errado (+1 obrigatório).
- **Levenshtein:** não tratar diferença de maiúsculas/minúsculas (use `toLowerCase()` antes de comparar).

Material de Apoio

- [Trie - GeeksforGeeks](#)
- [BST - GeeksforGeeks](#)
- [Distância de Levenshtein](#)
- [HashMap - Java Docs](#)
- [Dicionário de palavras em inglês](#)
- CORMEN, T. H. et al. *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: LTC, 2012.
- GOODRICH, M. T.; TAMASSIA, R. **Data Structures and Algorithms in Java**. 6. ed. New York: Wiley, 2014. 720 p. ISBN 978-1-118-77133-4.

Bom trabalho!